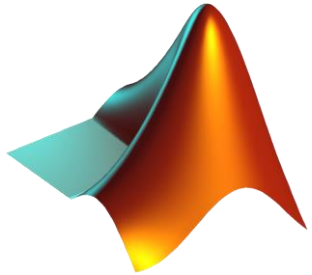
Politecnico di Torino

ROS Basics with MATLAB



P. David Cen Cheng
pangcheng.cencheng@polito.it
Andrea Usai
andea.usai@polito.it

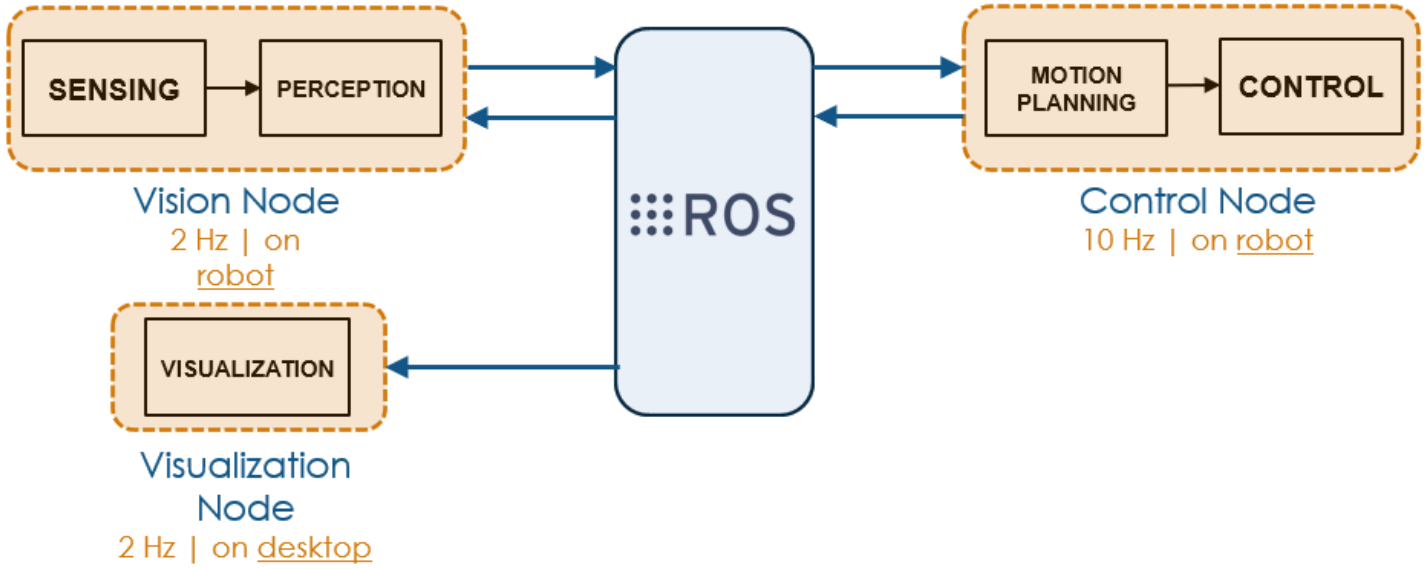
Robotics 2023/2024



Concepts

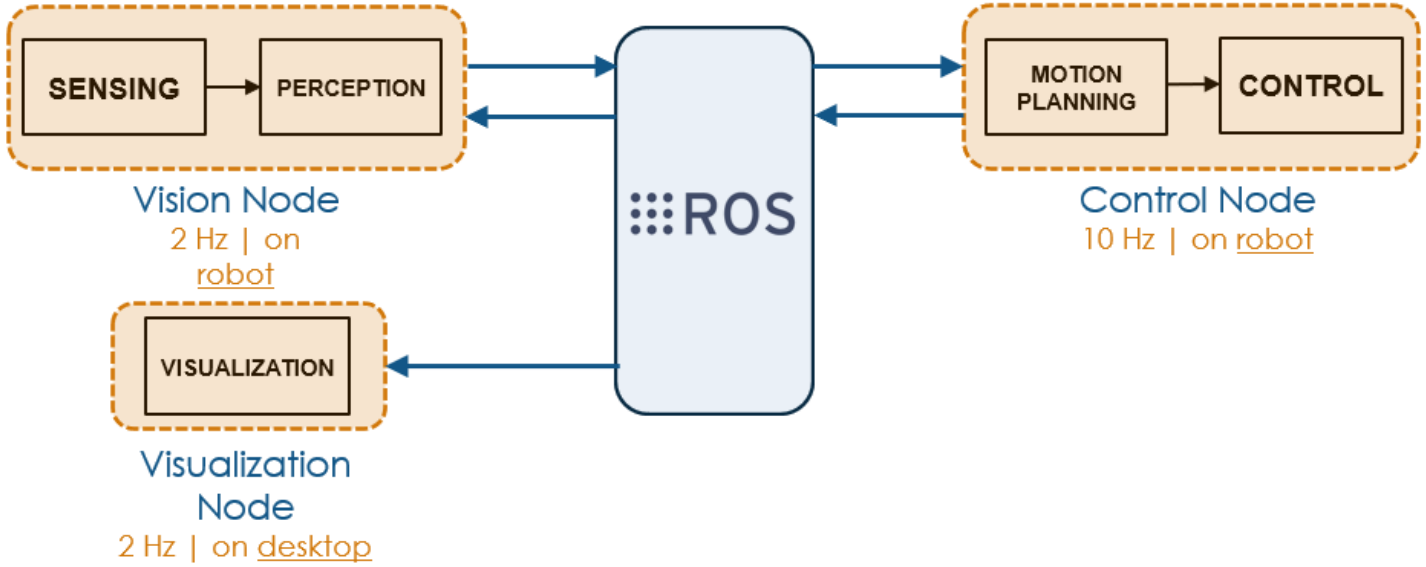
ROS network

- ROS network: it comprises different parts of a robot system (such as a planner or a camera interface) that communicate over ROS. The network can be distributed over several machines.



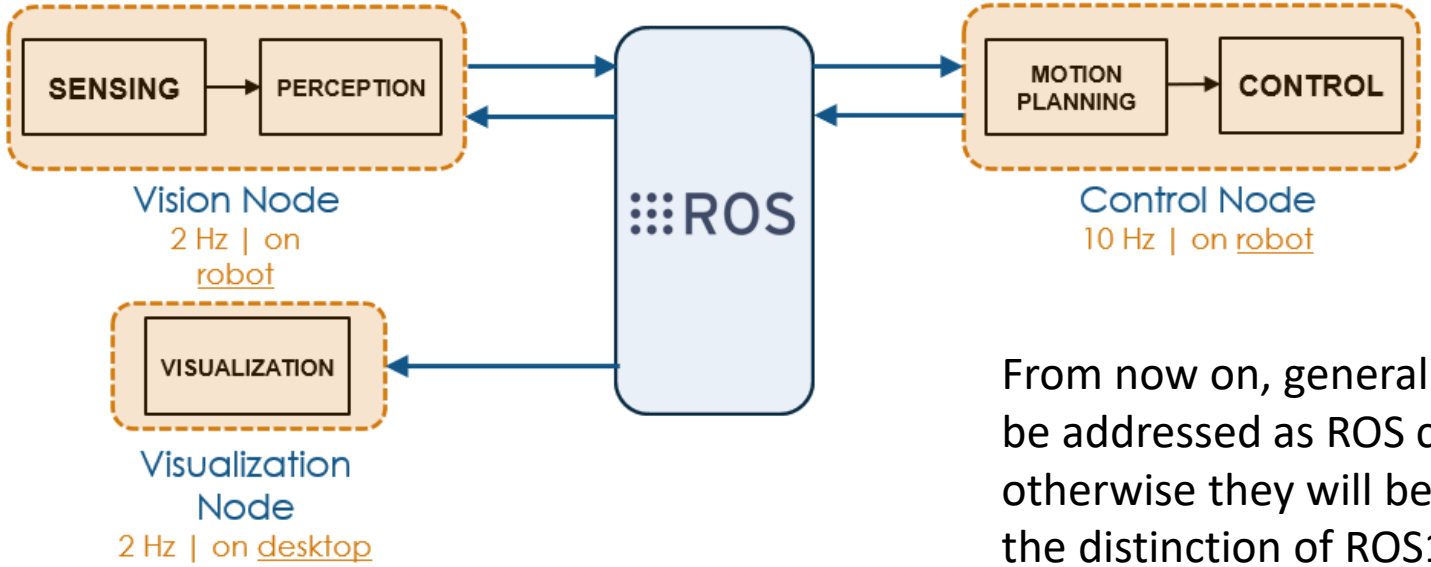
ROS Master

- ROS Master (only for ROS1): it coordinates the different parts of a ROS network. It is identified by a Master URI (Uniform Resource Identifier) that specifies the hostname or IP address of the machine where the master is running.



ROS Master

- ROS Master (only for ROS1): it coordinates the different parts of a ROS network. It is identified by a Master URI (Uniform Resource Identifier) that specifies the hostname or IP address of the machine where the master is running.



From now on, general concepts will be addressed as ROS concepts, otherwise they will be addressed with the distinction of ROS1 or ROS2

ROS Nodes and topics

- Each complex task can be split into simpler problems: in ROS simple tasks are solved using single purpose programs able to communicate through special channels
- In ROS, each running program is called node and the inter-communication channels are topics



ROS Nodes

- ROS node: A node is a process that performs computation
  - Each node can be a publisher node (on one or several topics) or a subscriber node
  - Nodes can publish/subscribe on/to one or several topics, as defined in its code
- These nodes are meant to operate at a fine-grained scale; a robot control system will usually comprise many nodes
  - For example, one node controls a laser range-finder, one Node controls the robot's wheel motors, one node performs localization, one node performs path planning, one node provides a graphical view of the system, and so on

ROS Nodes

- ROS node: A node is a process that performs computation
  - Each node can be a publisher node (on one or several topics) or a subscriber node
  - Nodes can publish/subscribe on/to one or several topics, as defined in its code
- A node in ROS1 is written with the use of a ROS client library, such as roscpp or rospy

ROS Topics

- Topic: communication channel among nodes, characterized by a message type

- A topic starts existing when a node declares it wants to publish on or subscribe to it. Topics should be used for continuous data streams, like sensor data, robot state, etc.

- Nodes can also communicate using Services and Actions. Topic mechanism is asynchronous

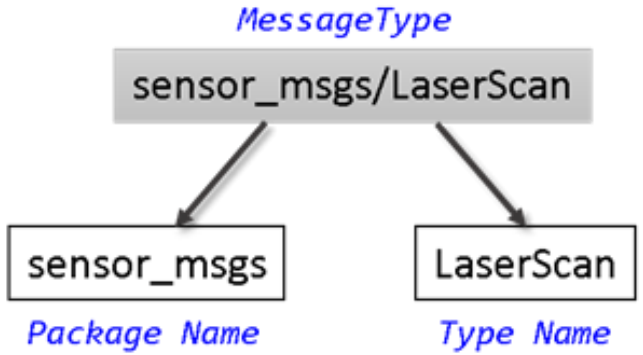
ROS Messages

- Messages: Data structures containing fields of predetermined types

- Messages are the primary container for exchanging data in ROS. Topics and services use messages to carry data between nodes

ROS Messages

- To identify its data structure, each message has a message type. For example, sensor data from a laser scanner is typically sent in a message of type sensor_msgs/LaserScan. Each message type identifies the data elements that are contained in a message. Every message type name is a combination of a package name, followed by a forward slash /, and a type name



ROS Namespacing

- ROS Computation Graph elements follow a hierarchical naming structure
- Each name is defined within a namespace
- Example: node1 publishes on “topic1” → the name of topic 1 is resolved to node1/topic1

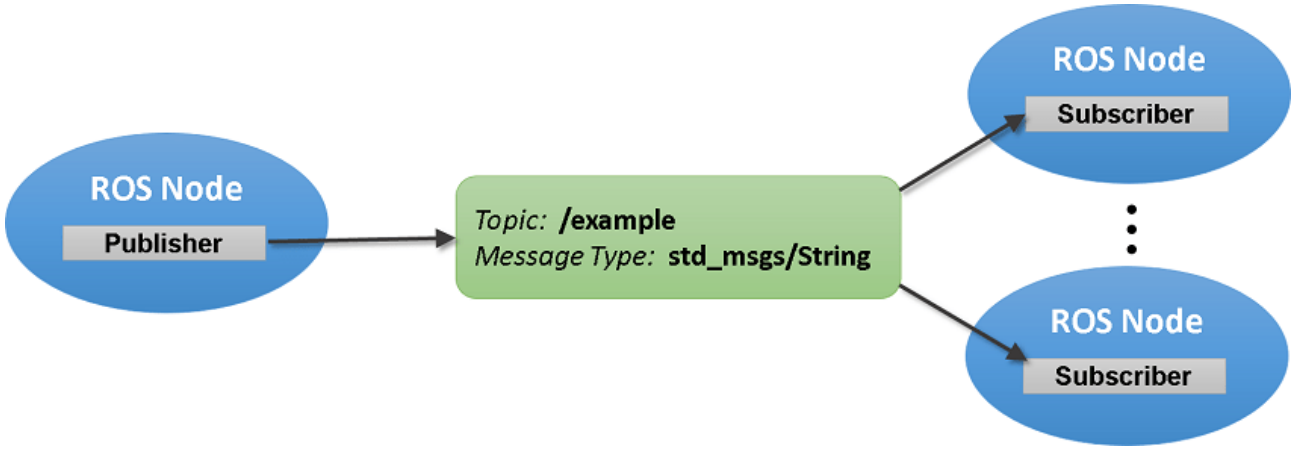
ROS Publishers and Subscribers

- The primary mechanism for ROS nodes to exchange data is sending and receiving messages. Messages are transmitted on a topic, and each topic has a unique name in the ROS network.

- If a node wants to share information, it uses a publisher to send data to a topic. A node that wants to receive that information uses a subscriber to that same topic. Besides its unique name, each topic also has a message type, which determines the types of messages that are capable of being transmitted under that topic.

ROS Publishers and Subscribers

- The publisher and subscriber communication has the following characteristics:
  - Topics are used for many-to-many communication. Many publishers can send messages to the same topic and many subscribers can receive them.
  - Publishers and subscribers are decoupled through topics and can be created and destroyed in any order. A message can be published to a topic even if there are no active subscribers.



ROS Tutorial

The tutorials use prerecorded data from MATLAB:

```
>> exampleHelperROSCreateSampleNetwork
```

For convenience, those files have been packed in the folder "ROS1_MATLAB_functions". Add that folder path to MATLAB.

- Tutorial 1: Understanding Nodes
- Tutorial 2: Understanding Topics
- Tutorial 3: Exchange Data with ROS Publishers and Subscribers
- Tutorial 4: Work with Basic ROS messages
- Tutorial 4.1: Work with Specialized ROS Messages

ROS Tutorial

- To initialize a ROS Master node: >> rosinit
  - It creates a ROS Master node, whose behaviour is similar to the ROS Core in Linux environment. It's the node of reference for the whole system.
- To shut down the ROS Master node: >> rosshutdown
  - It closes the Master node and all the processes that depends on that Master
- Nodes can be explored by typing:

```
>> rosnode list
>> rosnode info /node_name
>> rosnode ping /node_name
```

- Topics can be explored by typing:

```
>> rostopic list
>> rostopic info /topic_name
>> rostopic type /topic_name
>> rostopic echo /topic_name
```

ROS Tutorial

- Nodes can be publishers or subscribers
- Subscriber syntax:

```
>> sub = rossubscriber(“/topic_name”, “DataFormat”, “struct”)
>> msg = receive (sub)   % It waits to receive a topic from a subscriber
```

  - The “struct” format allows to plot the saved data
  - A callback can be used to process the data of the subscriber to avoid waiting using “receive”

```
>> sub = rossubscriber("/topic_name",@Callback,"DataFormat","struct")
```

- Publisher syntax:

```
>> pub = rospublisher(“/topic_name”, “message_Type”,“DataFormat”, “struct”)
>> msg = rosmessage(pub)   % initializes the message variable to be sent
>> msg.Data = ‘my_message’  % message to be sent
>> send(pub,msg)  % It sends the message through the publisher
```

The message is published in the node that subscribe that information

ROS Tutorial

- Messages in ROS: >> msg = rosmessage(“message_type”, “DataFormat”, “struct”)
- Message type in ROS:

std_msgs/String	For generic string data
sensor_msgs/Image	For image data
sensor_msgs/CompressedImage	
sensor_msgs/LaserScan	For Laser range finder data
sensor_msgs/PointCloud2	For Point cloud data
geometry_msgs/Quaternion	For position/pose/velocity data
geometry_msgs/TransformStamped	
nav_msgs/OccupancyGrid	For navigation data
octopmap_msgs/Octomap	For manipulation data
.	.
.	.
.	.

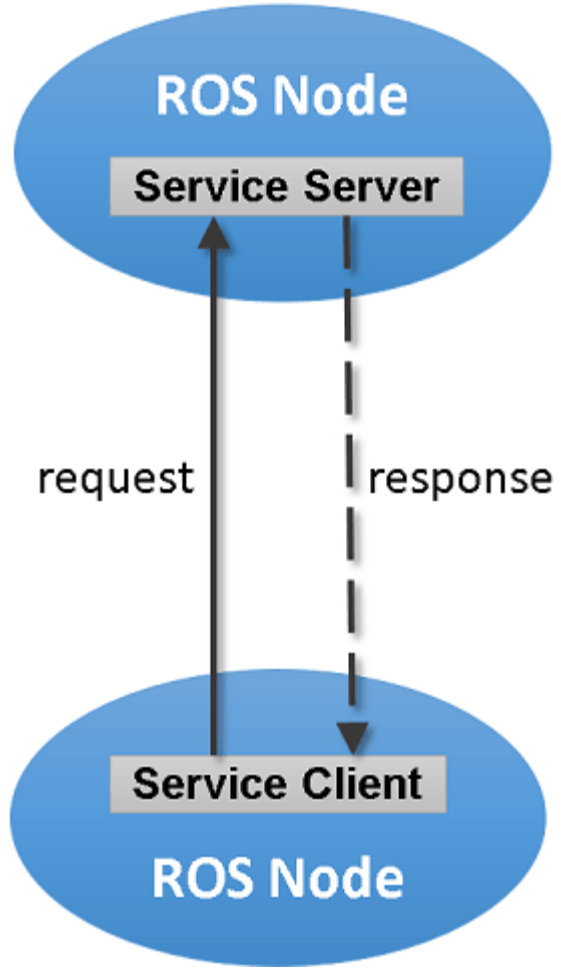
ROS Tutorial

- Messages in ROS: >> msg = rosmessage(“message_type”, “DataFormat”, “struct”)
- Message type in ROS:

Message Type	Structure Function
All messages	rosShowDetails
	rosmsg("show”,____)
sensor_msgs/Image sensor_msgs/CompressedImage	rosReadImage
sensor_msgs/Image	rosWriteImage
sensor_msgs/LaserScan	rosReadCartesian
	rosReadScanAngles
	rosReadLidarScan
	rosPlot
sensor_msgs/PointCloud2	rosReadXYZ
	rosReadRGB
	rosReadAllFieldNames
	rosReadField
	rosPlot
geometry_msgs/Quaternion	rosReadQuaternion
nav_msgs/OccupancyGrid	rosReadBinaryOccupancyGrid
	rosReadOccupancyGrid
	rosWriteBinaryOccupancyGrid
	rosWriteOccupancyGrid
octomap_msgs/Octomap	rosReadOccupancyMap3D
geometry_msgs/TransformStamped	rosApplyTransform

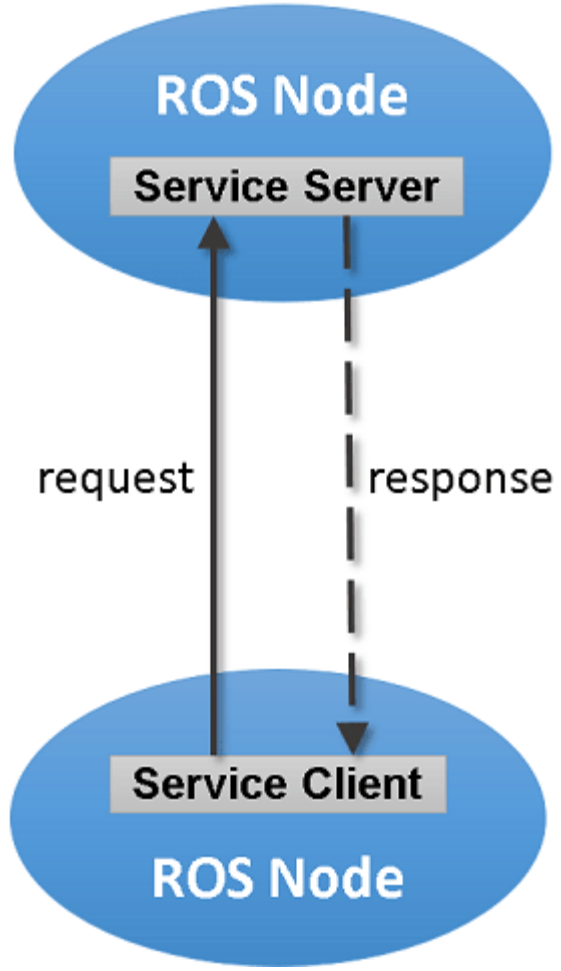
ROS Services

- Service: is a communication channel among nodes, characterized by a service type. A service type definition contains the message type for the request and the message type for the response, using a "Request/Reply" approach



ROS Services

- While topics should be used for continuous data streams, services should be used for quick requests, for example, for querying the state of a node or receive the result of a computation. Generally used when the algorithm will "wait" for a response



ROS Parameter Server

- Parameters are stored by nodes on a server, called Parameter Server
- It is used to store data that you want to access ‘globally’ from your ROS code. All the nodes use this server to store and retrieve parameters at runtime
- It is best used for static, non-binary data such as configuration parameters. It runs inside of the ROS Master

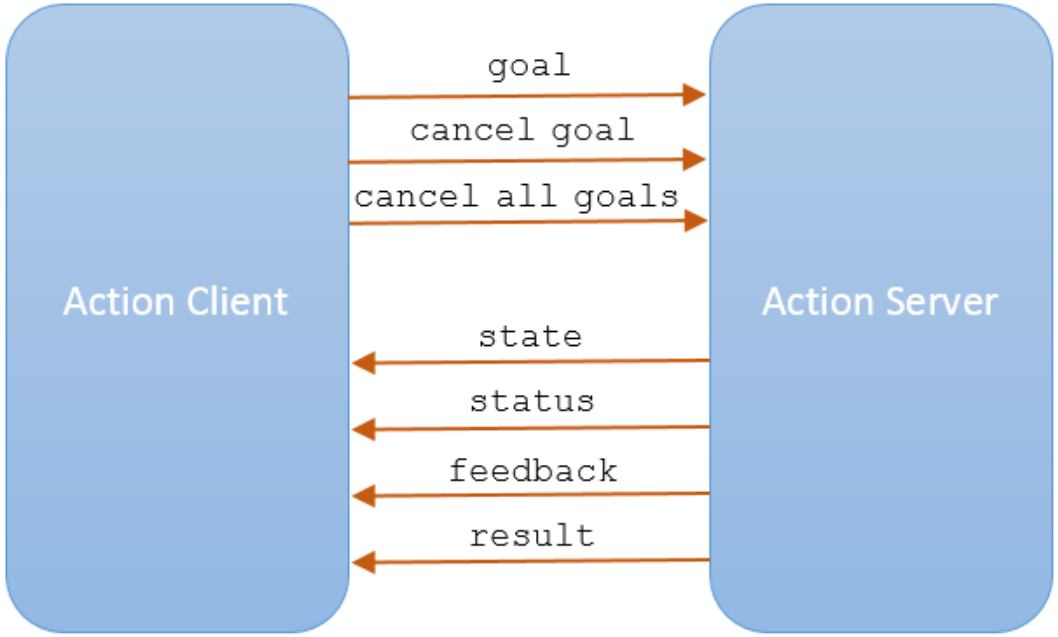
ROS Parameter Server

- Parameters are named following ROS “namespacing” conventions. This hierarchy is meant to protect parameter names from colliding

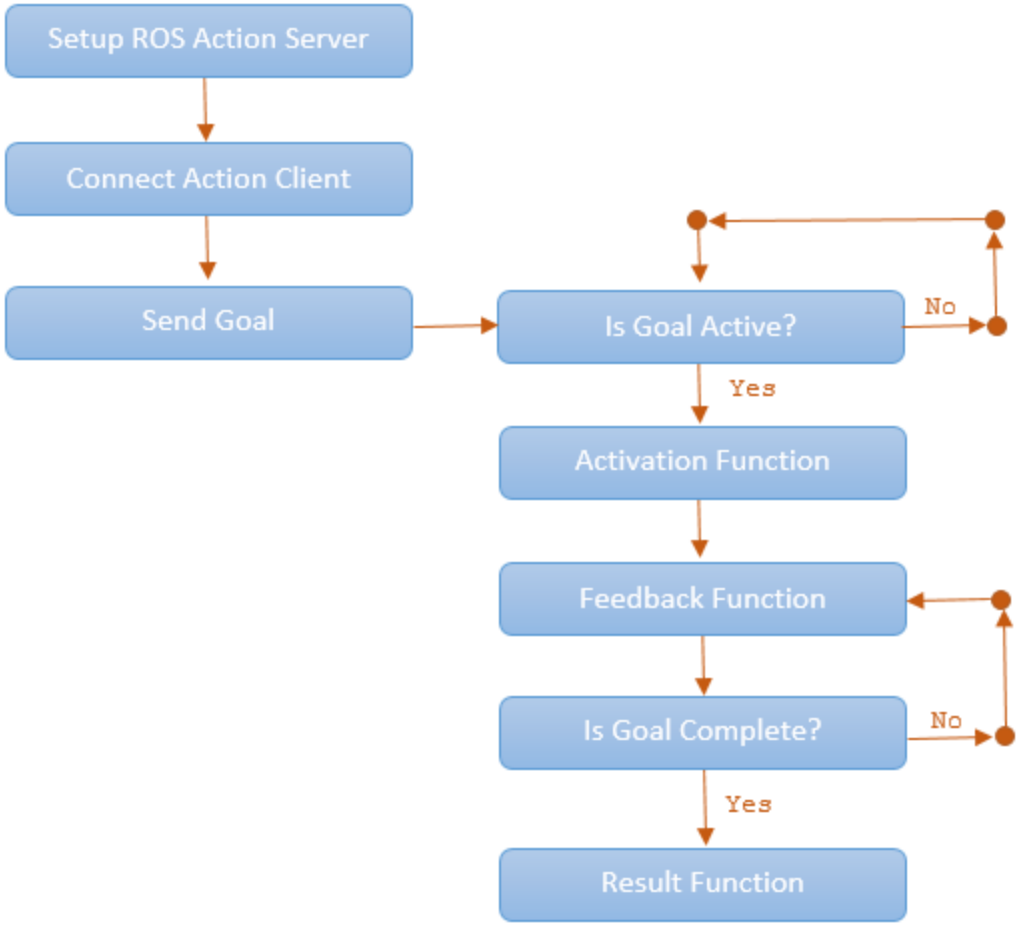


ROS Actions

- Actions are used for synchronous communication: user might want the ability to cancel the request during execution or get periodic feedback about how the request is progressing
- Actions use topics to send goal messages from a client to the server



ROS Actions



1. **rosactionserver** is used to set up the ROS action server.
2. **rosactionclient** is used to create action clients and connect them to the server. You can use waitForServer to wait for the action client to connect to the server.
3. Goal is sent using **sendGoal**. Define a **goalMsg** that corresponds to the action type.
4. When a goal status becomes 'active', the goal begins execution and the ActivationFcn callback function is called.
5. While the goal status remains 'active', the server continues to execute the goal. The feedback callback function processes information about this goals execution periodically whenever a new feedback message is received.
6. When the goal is achieved, the server returns a result message and status.

ROS Tutorial

- Tutorial 5: Understanding Services
- Tutorial 6: Understanding Parameter servers
- Tutorial 7: Understanding Action server

ROS Tutorial

- To define a service, we need to initialize a server and a client:

  >> server = rossvcserver(“/service_name”, “service_type”, @callback, “DataFormat”, “struct”)
  >> client = rossvcclient(“/service_name”, “DataFormat”, “struct”)

- A message can be transmitted using services through the call function

  >> msg_req= rosmessage (client)
  >> resp=call(client,msg_req, “Timeout”, 5)

- The parameter server can be explored through the rosparam function

  >> ptree = rosparam

ROS Tutorial

- The definition of the action server is similar to the services: needs a client and a server

```
>> cb=@callback_function
>> server = rosactionserver(“/action_name”, “action_type”, “ExecuteGoalFcn”, cb, "DataFormat","struct")
>> client = rosactionclient (“/action_name”, “action_type”, "DataFormat","struct")
>> goal = rosmessage(client);
>> goal.Order = int32(10);
>> result = sendGoalAndWait(client,goal);
```

ROS Topics vs Services vs Actions

Topics

- Should be used for continuous data streams (sensor data, robot state, etc).

- Are for continuous data flow. Data might be published and subscribed at any time independent of any senders/receivers. Many to many connection. Callbacks receive data once it is available. The publisher decides when data is sent.

ROS Topics vs Services vs Actions

Services

- Should be used for remote procedure calls that terminate quickly, e.g. for querying the state of a node or doing a quick calculation such as IK (inverse kinematics).
  - They should never be used for longer running processes, in particular processes that might be required to preempt if exceptional situations occur and they should never change or depend on state to avoid unwanted side effects for other nodes.

- Simple blocking call. Mostly used for comparably fast tasks as requesting specific data. Semantically for processing requests.

ROS Topics vs Services vs Actions

Actions

- Should be used for any discrete behavior that moves a robot or that runs for a longer time but provides feedback during execution.
- The most important property of actions is that they can be preempted and preemption should always be implemented cleanly by action servers.
- Actions can keep state for the lifetime of a goal, i.e. if executing two action goals in parallel on the same server, for each client a separate state instance can be kept since the goal is uniquely identified by its ID.

ROS Topics vs Services vs Actions

Actions

- Slow perception routines which take several seconds to terminate or initiating a lower-level control mode are good use cases for actions.
- More complex non-blocking background processing. Used for longer tasks like execution of robot actions. Semantically for real-world actions.

Note: preemption is the act of temporarily interrupting an executing task, with the intention of resuming it at a later time. This interrupt is done by an external scheduler with no assistance or cooperation from the task.

ROS Going beyond

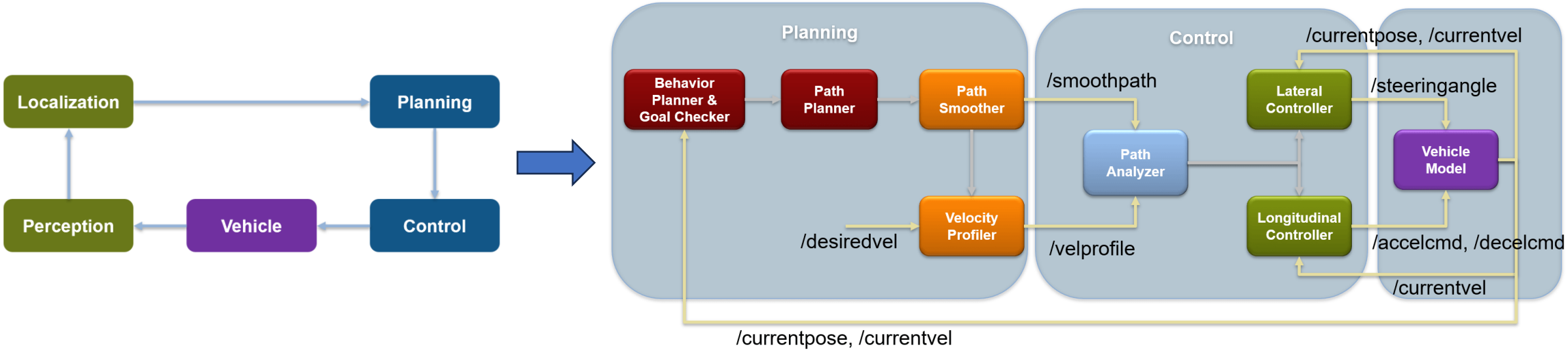
- Automated Parking Valet with ROS in MATLAB

This example puts together, in a simple way, the ingredients needed for an automated parking using MATLAB:
>> openExample('ros/AutomatedParkingValetWithROSInMATLABExample')

The vehicle's automated systems are expected to take over control and steer the vehicle to an available parking spot. Such a function makes use of multiple on-board sensors. For example:

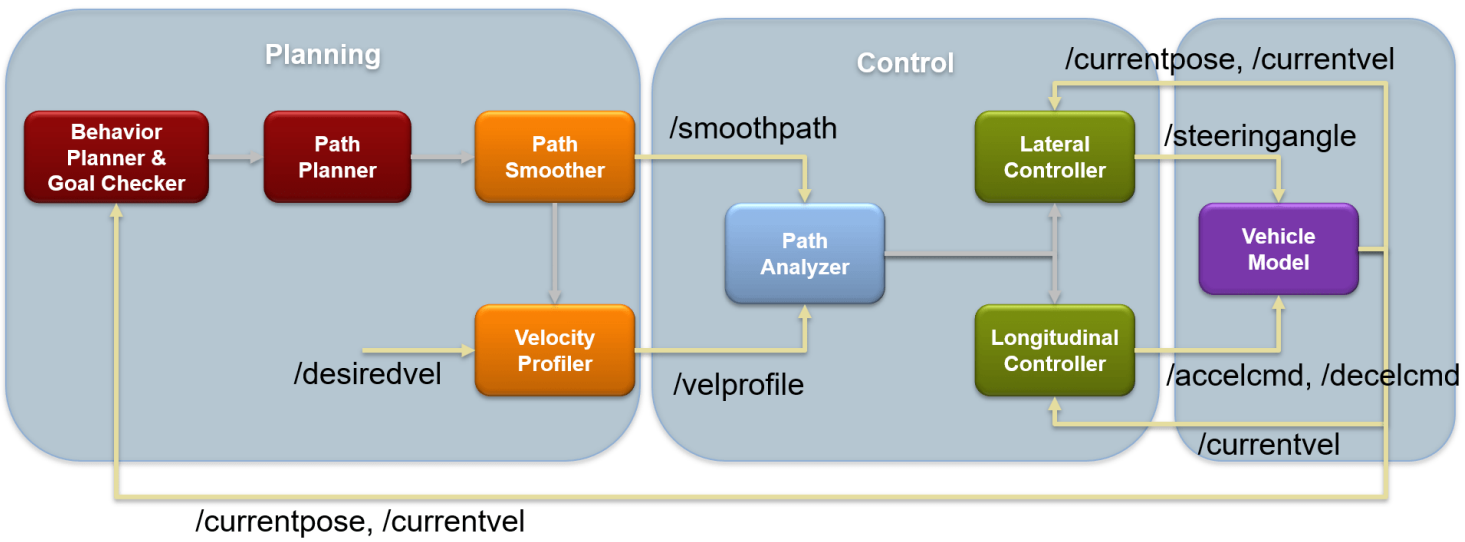
- Front and side cameras for detecting lane markings, road signs (stop signs, exit markings, etc.), other vehicles, and pedestrians
- Lidar and ultrasound sensors for detecting obstacles and calculating accurate distance measurements
- Ultrasound sensors for obstacle detection
- IMU and wheel encoders for dead reckoning

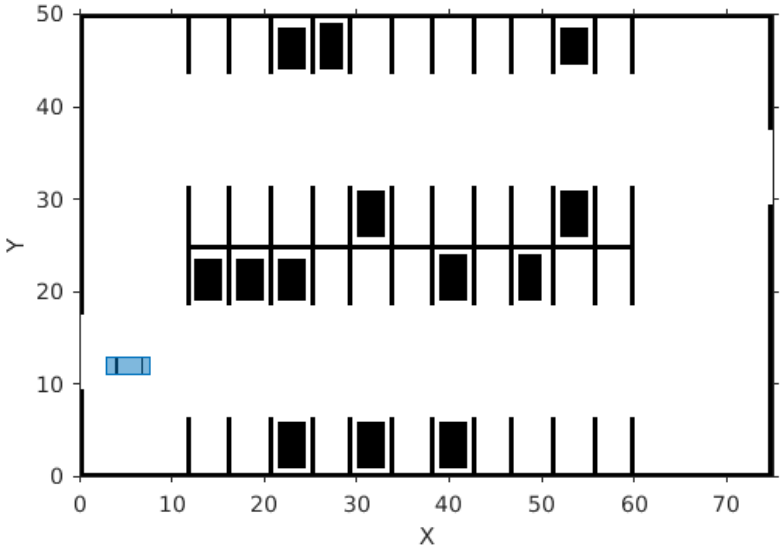
Automated Parking Valet with ROS in MATLAB



For simplicity, this example uses prerecorded data to substitute localization information

Automated Parking Valet with ROS in MATLAB

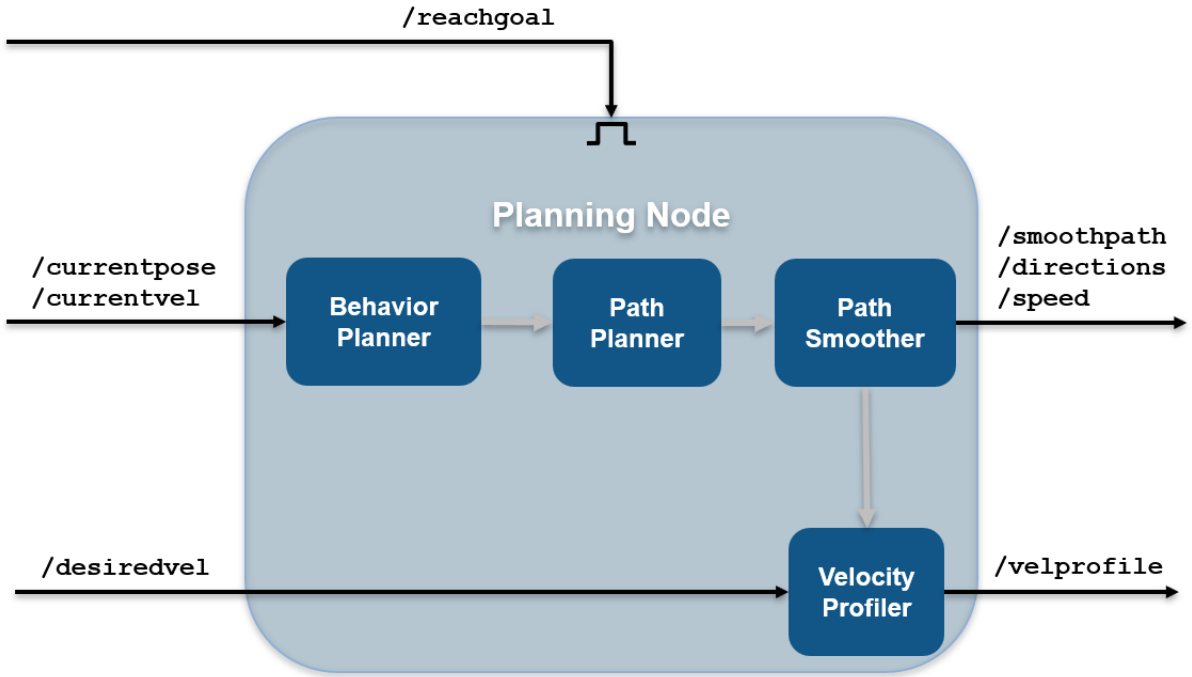




For simplicity, this example uses prerecorded data to substitute localization information

Automated Parking Valet with ROS in MATLAB

- Planning node:



Publishes to these topics

- /smoothpath
- /velprofile
- /directions
- /speed
- /nextgoal

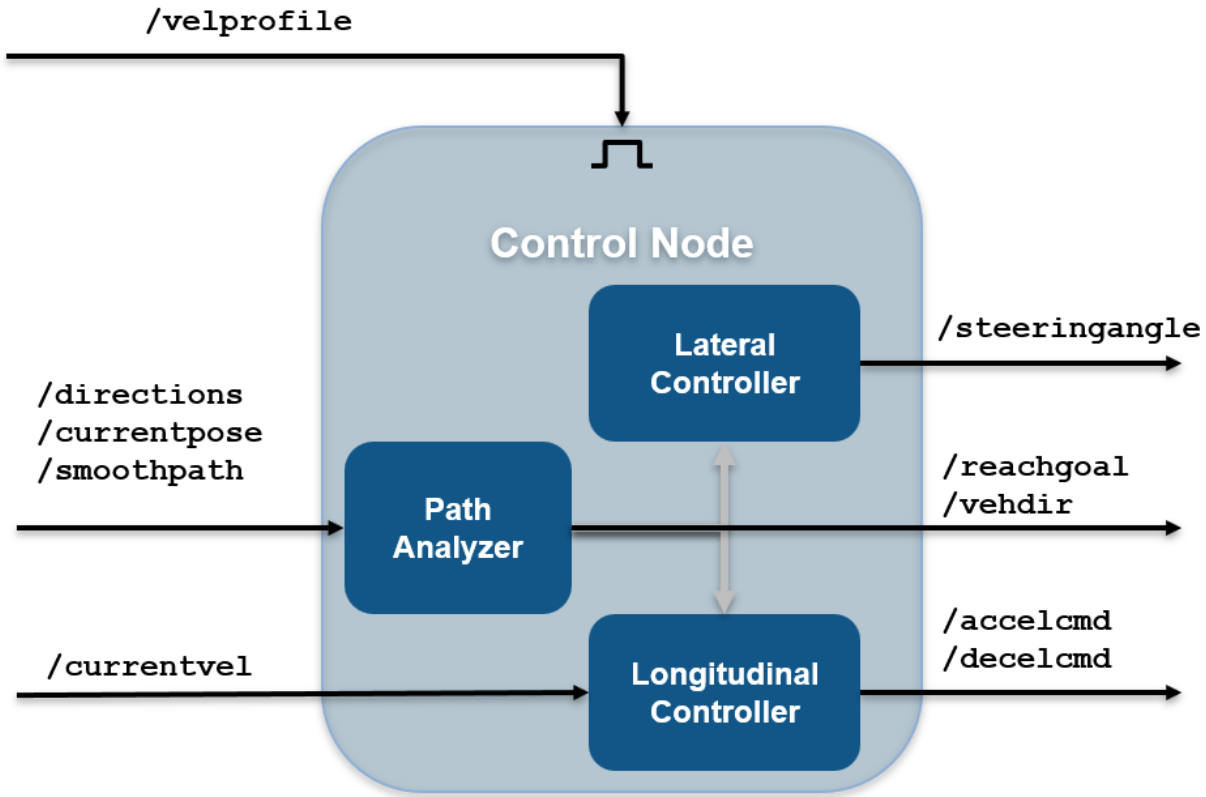
Subscribes to these topics

- /currentvel
- /currentpose
- /desiredvel
- /reachgoal

On receiving a /reachgoal message, the node runs the **exampleHelperROS2ValetPlannerCallback** callback, which plans the next segment.

Automated Parking Valet with ROS in MATLAB

- Control node:



Publishes to these topics

- /steeringangle
- /accelcmd
- /decelcmd
- /vehidir
- /reachgoal

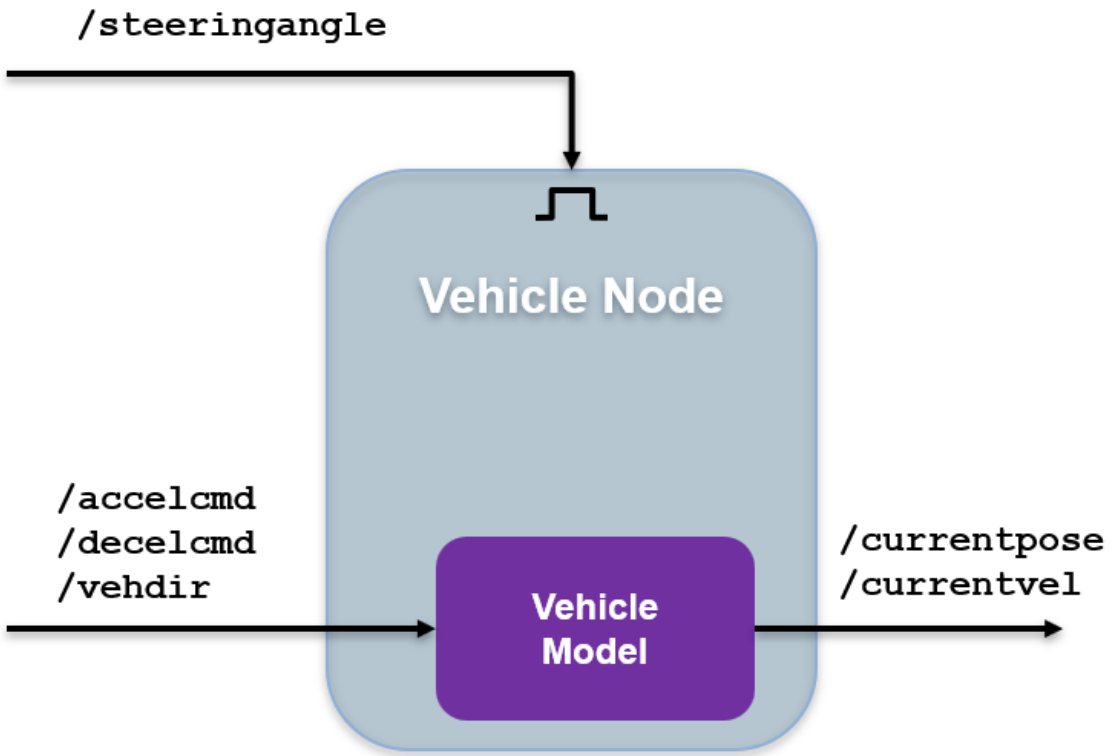
Subscribes to these topics

- /smoothpath
- /directions
- /currentvel
- /currentpose
- /desiredvel
- /nextgoal
- /velprofile

On receiving a /velprofile message, the node runs the **exampleHelperROS2ValetControlCallback** callback, which sends control messages to the vehicle

Automated Parking Valet with ROS in MATLAB

- Control node:



Publishes to these topics
- /currentvel
- /currentpose

Subscribes to these topics
- /steeringangle
- /accelcmd
- /decelcmd
- /vehidir

On receiving a /steeringangle message, the vehicle simulator is run in the callback function, **exampleHelperROSValetVehicleCallback**.

ROS Tutorial References

- Tutorial 1: Understanding Nodes
- Tutorial 2: Understanding Topics
- Tutorial 3: Exchange Data with ROS Publishers and Subscribers
- Tutorial 4: Work with Basic ROS messages
- Tutorial 4.1: Work with Specialized ROS Messages
- Tutorial 5: Understanding Services
- Tutorial 6: Understanding Parameter servers
- Tutorial 7: Understanding Action servers
- Automated Parking Valet with ROS in MATLAB